Лабораторная работа 5 "Взаимодействие React-приложения с сервером"

Лабораторная работа №5. Взаимодействие React-приложения с сервером

Ветка репозитория: `25`

---

1. ЦЕЛЬ РАБОТЫ

Освоение методов отправки HTTP-запросов из React-приложения к REST API, обработки асинхронных состояний (загрузка, ошибка, успех) и обновления интерфейса на основе полученных данных **для интеграции клиентской части вашего курсового проекта с серверным API**.

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Клиент-серверное взаимодействие, REST API.
- Fetch API и библиотеку axios.
- Асинхронные функции, `async/await`.
- Состояния загрузки (loading) и ошибки (error) в React.
- Оптимистичное обновление UI.

---

3. ЗАДАЧИ РАБОТЫ

- Взять React-приложение, созданное в лабораторной работе №4 (или новое). Оно должно взаимодействовать с **вашим собственным API**, разработанным в лабораторных работах №1, №2, №3 (сервер Node.js + Express + PostgreSQL + аутентификация JWT).

- Реализовать получение списка элементов с сервера (GET) с отображением загрузки и ошибки.
- Реализовать добавление нового элемента (POST) с оптимистичным обновлением.
- Реализовать удаление элемента (DELETE) с обновлением списка.
- Реализовать редактирование элемента (PUT/PATCH).
- Добавить **одно дополнительное улучшение** (пагинацию, поиск, debounce, повтор запросов, отмену запроса при размонтировании и т.п.) в соответствии с логикой вашего курсового проекта.

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Настройка проекта

1. Убедитесь, что ваш сервер (из лабораторных №1–№3) запущен и доступен по адресу (например, `http://localhost:3000`).
2. В React-проекте, созданном в лабораторной работе №4, установите axios (или используйте fetch):

```
npm install axios
```

3. Настройте переменные окружения (`.env`):

```
VITE_API_URL=http://localhost:3000/api
```

Часть 2. Создание сервиса API

Создайте файл `src/api.js` с функциями для взаимодействия с вашим API. Функции должны использовать JWT-токен (если аутентификация обязательна) и обрабатывать ошибки.

Пример с использованием axios:

```
import axios from 'axios';

const api = axios.create({
```

```
  baseURL: import.meta.env.VITE_API_URL,
});

// Перехватчик для добавления токена
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export const fetchItems = () => api.get('/items');
export const addItem = (item) => api.post('/items', item);
export const updateItem = (id, updated) => api.put(`/items/${id}`, updated);
export const deleteItem = (id) => api.delete(`/items/${id}`);
```

Часть 3. Компонент списка

- Используйте `useState` для хранения массива данных, флага `loading` и `error`.
- В `useEffect` (с пустым массивом зависимостей) загружайте данные вызовом `fetchItems()`.
- При загрузке отображайте индикатор, при ошибке – сообщение и кнопку «Повторить».

Часть 4. Реализация CRUD-операций с оптимистичным обновлением

- **Добавление:**
  - сразу добавить новый элемент в локальное состояние (с временным ID, например `Date.now()`);
  - отправить POST-запрос;
  - при успехе заменить временный ID на реальный из ответа сервера;
  - при ошибке – удалить элемент из состояния и показать уведомление.
- **Удаление:**
  - оптимистично убрать элемент из списка;
  - отправить DELETE;
  - при ошибке – вернуть элемент обратно.
- **Редактирование:**

- оптимистично обновить элемент в списке;
- отправить PUT/PATCH;
- при ошибке – откатить изменение.

Часть 5. Дополнительное улучшение (по выбору)

Выберите **одно** из следующих улучшений (или другое, согласованное с преподавателем):

- **Пагинация** – загрузка элементов постранично (limit/offset) с кнопками «Загрузить ещё» или навигацией.
- **Поиск (фильтрация) на сервере** – добавление эндпоинта с параметром `search` и debounce на ввод.
- **Повтор запроса при ошибке** (retry) – до 3 попыток с задержкой.
- **Отмена запроса при размонтировании** – с использованием `AbortController`.
- **Бесконечная прокрутка** (infinite scroll) – подгрузка следующей страницы при прокрутке.
- **Кэширование ответов** – с использованием `localStorage` и инвалидацией кэша.
- **Загрузка изображений** – через `FormData` (специальный эндпоинт на сервере).

Часть 6. Тестирование

- Запустите сервер (`npm run dev` в папке server) и React-приложение (`npm run dev` в папке client).
- Проверьте все CRUD-операции, состояния загрузки и ошибок, работу дополнительного улучшения.

---

5. СОДЕРЖАНИЕ ОТЧЕТА

Отчёт должен содержать:

- Титульный лист, цель работы.
- Схему взаимодействия React-приложения с сервером (текстовое описание).
- Листинг файла `api.js` и основного компонента (ключевые фрагменты кода).

- Скриншоты:
  - процесс загрузки данных (отображение спиннера);
  - список загруженных элементов;
  - добавление нового элемента (до и после);
  - удаление элемента;
  - сообщение об ошибке при недоступности сервера;
  - демонстрация выбранного дополнительного улучшения.
- Ответы на контрольные вопросы (16 вопросов).
- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- Ссылка на репозиторий GitHub с кодом (ветка **25**).
- **Индивидуальное выполнение. Вариант** **Темы** определяется по **Генератору вариантов.**
- Код должен быть загружен в репозиторий с веткой `25`.

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ (16 вопросов)

1. В чём разница между `fetch` и `axios`? Приведите преимущества и недостатки.
2. Как обработать состояние загрузки (loading) в React? Приведите пример из вашего кода.
3. Что такое оптимистичное обновление? Опишите пошагово на примере добавления элемента.
4. Как отобразить ошибку, если сервер вернул статус 500?
5. Зачем нужен `AbortController` при работе с запросами в `useEffect`? Как его использовать?
6. Как отправить POST-запрос с JSON-данными с помощью `fetch` и `axios`?

7. Как обработать ошибку сети (отсутствие интернета) в `fetch` и `axios`?
8. Как реализовать повторную попытку запроса при неудаче? Напишите псевдокод.
9. Как добавить `Authorization` header (Bearer token) к каждому запросу в axios с помощью interceptors?
10. Что такое дебаунс (debounce) и как он применяется для поискового ввода? Приведите пример.
11. Как отменить запрос при размонтировании компонента в `useEffect` с помощью `AbortController`?
12. Как имитировать задержку ответа сервера для тестирования состояний загрузки?
13. Как реализовать бесконечную прокрутку (infinite scroll) с пагинацией?
14. Как обрабатывать ошибки при оптимистичном обновлении и выполнять откат изменения?
15. Какие заголовки HTTP необходимо установить для отправки FormData (файла) на сервер?
16. Как в React отобразить индикатор загрузки при выполнении фонового запроса (например, при отправке данных)?

---

8. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Это ключевая лабораторная работа, связывающая клиентскую (React) и серверную (Node.js) части вашего курсового проекта.
- Убедитесь, что сервер поддерживает CORS. Если нет – добавьте middleware `cors` в Express (лабораторная работа №3).
- JWT-токен должен сохраняться после логина (например, в `localStorage`) и автоматически подставляться в запросы (см. пример с axios interceptor).

---

9. КРИТЕРИИ ОЦЕНКИ

- Реализация всех CRUD-операций (GET, POST, PUT/PATCH, DELETE) с сервером.
- Корректная обработка состояний загрузки и ошибок.

- Использование оптимистичного обновления и откат при ошибках.
- Наличие выбранного дополнительного улучшения (пагинация, поиск, retry, отмена запроса и т.п.).
- Качество кода: структура проекта, читаемость, комментарии.
- Полнота отчёта (скриншоты, листинги, ответы на вопросы).
- Ссылка на репозиторий с веткой `25` и инструкция по запуску.